

# UNIVERSIDAD AUTÓNOMA DE CHIAPAS

Facultad de Contaduría y Administración C-1

Ingeniería en Desarrollo y Tecnologías de Software

## Subsistema analítico y Dashboard de administración

Puerto de entrada **AnalyticsService**, puerto de salida **AnalyticsRepository**  
y panel interactivo en React + Recharts · E-commerce 5M

REPOSITORIO DEL CÓDIGO FUENTE

[github.com/LuisAGA-1/ecommerce-5m](https://github.com/LuisAGA-1/ecommerce-5m)

Alumno: **Luis Angel Alegria Gomez**

Semestre y grupo: 5.º semestre · Grupo M

Entorno de pruebas: Local (localhost) · WSL2 Ubuntu 24.04 · PostgreSQL

Fecha: 8 de octubre de 2026

Lugar: Tuxtla Gutiérrez, Chiapas

# 1. Objetivo y alcance

Extender el e-commerce con un subsistema analítico que procesa, agrega y visualiza métricas de rendimiento comercial, sin acoplar la lógica de reportes a las entidades operativas del dominio.

- **Volumen de ventas por producto:** Top 5 o Top 10 por unidades vendidas o por monto recaudado.
- **Ingresos totales:** sumatoria por rango de fechas, agrupada por día, semana o mes.
- **Estado de pedidos:** conteo y porcentaje en Pendiente, Pagado, Enviado y Cancelado.
- **Ticket promedio:** por pedido (ingresos ÷ pedidos) y por cliente (ingresos ÷ clientes que compraron), con variación frente al período anterior de igual duración.

El backend consulta el historial de **Usuario, Producto y Pedido** mediante un **puerto de consultas analíticas** separado de los repositorios operativos (CQRS ligero). Las agregaciones (SUM, COUNT, GROUP BY, filtros de fecha) se ejecutan en PostgreSQL, así que Node.js nunca carga pedidos individuales en memoria.

# 2. Arquitectura

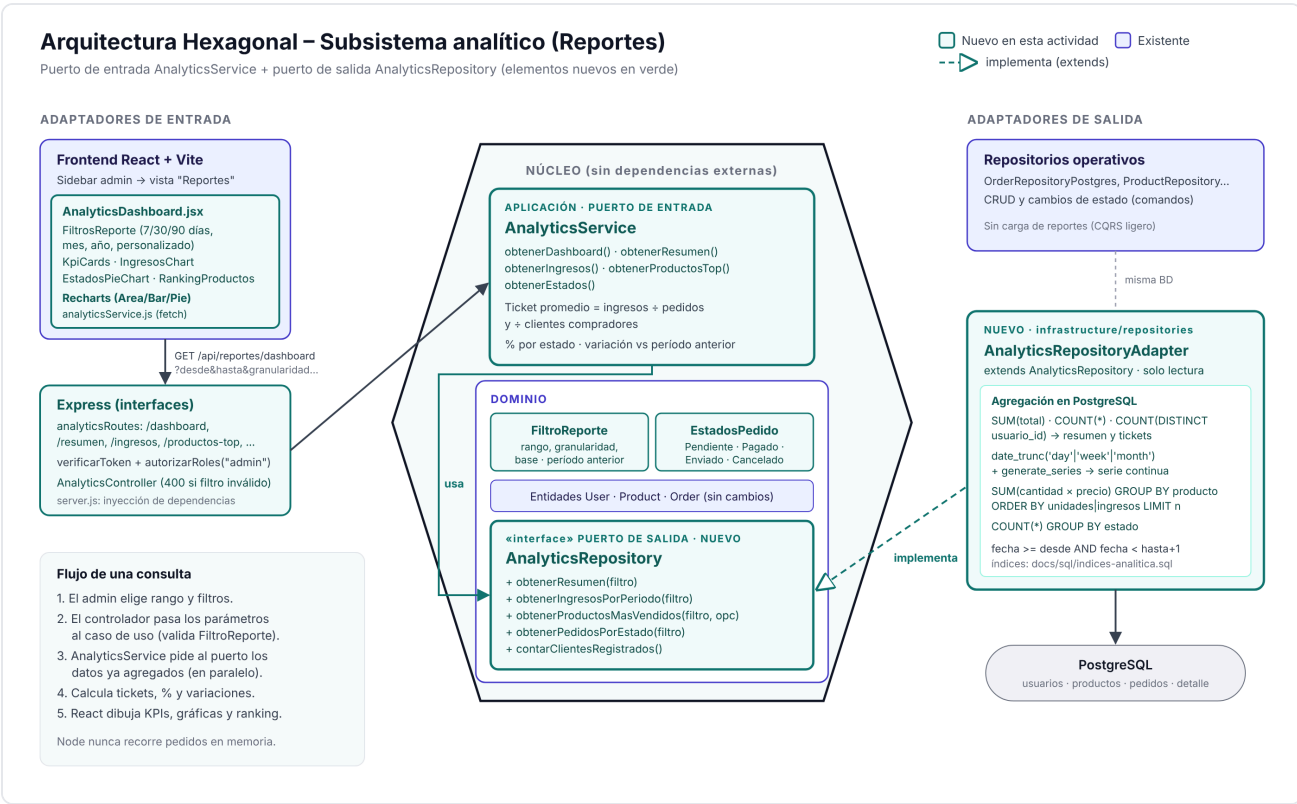


Figura 1. Subsistema analítico dentro de la arquitectura hexagonal (nuevos componentes en verde). Archivo: docs/arquitectura-analitica.svg

### 3. Entregable 1 – Especificación de endpoints analíticos

Base URL local: `http://localhost:3000/api/reportes`. Todos los endpoints requieren `Authorization: Bearer <JWT>` y rol **admin**; son de solo lectura (GET). La especificación completa, con ejemplos de respuesta, está en `docs/API-REPORTES.md`.

| Endpoint                          | Descripción                                                                                                    | Parámetros                                           |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| GET /api/reportes/dashboard       | Panel completo en una petición: resumen, tickets, variación, serie de ingresos, ranking y estados              | desde, hasta, granularidad, base, limite, ordenarPor |
| GET /api/reportes/resumen         | Ingresos, pedidos, clientes, unidades, ticket promedio por pedido y por cliente, variación vs período anterior | desde, hasta, base                                   |
| GET /api/reportes/ingresos        | Ingresos totales agrupados por día, semana o mes (serie continua con ceros)                                    | desde, hasta, granularidad, base                     |
| GET /api/reportes/productos-top   | Top N productos por unidades o por monto recaudado, con % de participación                                     | desde, hasta, base, limite, ordenarPor               |
| GET /api/reportes/estados-pedidos | Conteo, monto y % de pedidos: Pendiente, Pagado, Enviado, Cancelado                                            | desde, hasta                                         |

#### Parámetros (query string)

| Parámetro    | Valores             | Por defecto   | Descripción                                                             |
|--------------|---------------------|---------------|-------------------------------------------------------------------------|
| desde        | AAAA-MM-DD          | hoy – 29 días | Inicio del rango (inclusive)                                            |
| hasta        | AAAA-MM-DD          | hoy           | Fin del rango (inclusive)                                               |
| granularidad | día   semana   mes  | automática    | ≤31 días → día; ≤120 → semana; mayor → mes                              |
| base         | pagados   todos     | pagados       | pagados = ACEPTADO/ENVIADO; todos = además PENDIENTE (nunca cancelados) |
| limite       | 1-50                | 5             | Tamaño del ranking                                                      |
| ordenarPor   | unidades   ingresos | unidades      | Criterio del ranking                                                    |

Ejemplo: GET /api/reportes/dashboard?desde=2026-10-02&hasta=2026-10-08

```
{
  "rango": {
    "desde": "2026-10-02",
    "hasta": "2026-10-08",
    "dias": 7,
    "granularidad": "dia",
    "base": "pagados",
    "estadosConsiderados": [
      "ACEPTADO",
      "PAGADO",
      "ENVIADO",
      "ENTREGADO"
    ]
  },
  "resumen": {
    "ingresos": 118226,
    "pedidos": 10,
    "clientesCompradores": 5,
    "unidadesVendidas": 48,
    "ticketPromedioPedido": 11822.6,
    "ticketPromedioCliente": 23645.2,
    "clientesRegistrados": 7
  },
  "variacion": {
    "ingresos": -54.9,
    "pedidos": -9.1,
    "ticketPromedioPedido": -50.4,
    "ticketPromedioCliente": -54.9
  }
}
```

La respuesta completa incluye además `periodoAnterior`, `ingresos.serie` (un punto por período), `productosTop.productos` y `estados.estados`.

### Códigos de respuesta

| Código    | Situación                                                                                                                               |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------|
| 200       | Reporte generado.                                                                                                                       |
| 400       | Filtro inválido: fecha mal formada, "desde" posterior a "hasta", granularidad, límite u orden fuera de rango, o rango mayor a 731 días. |
| 401 / 403 | Sin token válido, o usuario sin rol admin.                                                                                              |
| 500       | Error de base de datos ("Error al generar el reporte").                                                                                 |

## 4. Entregable 2 – Código fuente backend y frontend

Repositorio: <https://github.com/LuisAGA-1/ecommerce-5m>. Archivos agregados y modificados:

| Archivo                                                                           | Capa            | Tipo       | Descripción                                                                  |
|-----------------------------------------------------------------------------------|-----------------|------------|------------------------------------------------------------------------------|
| backend/src/domain/por<br>ts/AnalyticsRepositor<br>y.js                           | Dominio         | Nuevo      | Puerto de salida de consultas analíticas (solo lectura).                     |
| backend/src/domain/ana<br>lytics/FiltroReporte.j<br>s                             | Dominio         | Nuevo      | Objeto de valor: valida rango, granularidad, base; calcula período anterior. |
| backend/src/domain/ana<br>lytics/EstadosPedido.j<br>s                             | Dominio         | Nuevo      | Mapea estados de BD a Pendiente/Pagado/Enviado/Cancelado.                    |
| backend/src/applicatio<br>n/analytics/AnalyticsS<br>ervice.js                     | Aplicación      | Nuevo      | Caso de uso: tickets, porcentajes, variaciones, ranking, dashboard.          |
| backend/src/infrastruc<br>ture/repositories/Anal<br>yticsRepositoryAdapte<br>r.js | Infraestructura | Nuevo      | Adaptador PostgreSQL con SUM, COUNT, GROUP BY, date_trunc, generate_series.  |
| backend/src/interface<br>s/controllers/analytic<br>sController.js                 | Interfaces      | Nuevo      | Adaptador HTTP; errores de filtro → 400.                                     |
| backend/src/interface<br>s/routes/analyticsRout<br>es.js                          | Interfaces      | Nuevo      | Rutas /api/reportes/* con JWT y rol admin.                                   |
| backend/src/interface<br>s/server.js                                              | Interfaces      | Modificado | Composición del módulo analítico.                                            |
| backend/tests/analitic<br>a.test.js                                               | Pruebas         | Nuevo      | 13 pruebas del dominio y del caso de uso con repositorio falso.              |
| backend/scripts/seedAn<br>alitica.js                                              | Script          | Nuevo      | Datos demo con un solo INSERT...SELECT (y -- limpiar).                       |
| docs/sql/indices-anali<br>tica.sql                                                | BD              | Nuevo      | Índices recomendados para los filtros de reportes.                           |
| frontend/src/modules/a<br>nalytics/AnalyticsDash<br>board.jsx                     | Frontend        | Nuevo      | Vista del panel: estado, filtros y carga de datos.                           |

|                                                             |          |              |                                                                             |
|-------------------------------------------------------------|----------|--------------|-----------------------------------------------------------------------------|
| frontend/src/modules/analytics/components/*.jsx             | Frontend | Nuevo        | FiltrosReporte, KpiCards, IngresosChart, EstadosPieChart, RankingProductos. |
| frontend/src/services/analyticsService.js                   | Frontend | Nuevo        | Cliente HTTP de /api/reportes.                                              |
| frontend/src/App.jsx · Sidebar.jsx · App.css · package.json | Frontend | Modificado   | Vista "Reportes" en el menú admin, estilos y dependencia recharts.          |
| docs/API-REPORTES.md · README.md                            | Docs     | Nuevo / Mod. | Especificación de endpoints y guía de ejecución.                            |

## 4.1 Puerto de salida (dominio)

```
class AnalyticsRepository {

    /**
     * Totales del período.
     * @returns {Promise<{ pedidos:number, ingresos:number, clientes:number, unidades:number }>}
     */
    async obtenerResumen(filtro) {
        throw new Error("Método obtenerResumen no implementado");
    }

    /**
     * Ingresos agrupados por día/semana/mes, incluyendo períodos en cero.
     * @returns {Promise<Array<{ periodo:string, ingresos:number, pedidos:number }>>}}
     */
    async obtenerIngresosPorPeriodo(filtro) {
        throw new Error("Método obtenerIngresosPorPeriodo no implementado");
    }

    /**
     * Ranking de productos.
     * @param {Object} opciones { limite:number, ordenarPor:'unidades'|'ingresos' }
     * @returns {Promise<Array<{ productoId:number, nombre:string, unidades:number, ingresos:number,
    pedidos:number }>>}}
     */
    async obtenerProductosMasVendidos(filtro, opciones) {
        throw new Error("Método obtenerProductosMasVendidos no implementado");
    }

    /**
     * Conteo y monto por estado de BD (sin filtrar por estado).
     * @returns {Promise<Array<{ estado:string, cantidad:number, monto:number }>>}}
     */
    async obtenerPedidosPorEstado(filtro) {
        throw new Error("Método obtenerPedidosPorEstado no implementado");
    }

    /**
     * Total de clientes registrados (entidad Usuario).
     * @returns {Promise<number>}}
     */
    async contarClientesRegistrados() {
        throw new Error("Método contarClientesRegistrados no implementado");
    }
}
```

## 4.2 Adaptador PostgreSQL – ingresos por período

date\_trunc agrupa por día, semana o mes, y generate\_series rellena los períodos sin ventas, así la gráfica sale continua.

```
async obtenerIngresosPorPeriodo(filtro) {
  const { rows } = await this.db().query(
    `WITH serie AS (
      SELECT generate_series(
        date_trunc($4, $1::date::timestamp),
        date_trunc($4, $2::date::timestamp),
        ('1 ' || $4)::interval
      ) AS periodo
    ),
    ventas AS (
      SELECT
        date_trunc($4, p.fecha::timestamp) AS periodo,
        SUM(p.total) AS ingresos,
        COUNT(*) AS pedidos
      FROM pedidos p
      WHERE ${FILTRO_PEDIDOS}
      GROUP BY 1
    )
    SELECT
      to_char(s.periodo, 'YYYY-MM-DD') AS periodo,
      COALESCE(v.ingresos, 0) AS ingresos,
      COALESCE(v.pedidos, 0)::int AS pedidos
    FROM serie s
    LEFT JOIN ventas v ON v.periodo = s.periodo
    ORDER BY s.periodo`,
    [filtro.desde, filtro.hasta, filtro.estados, filtro.unidadSql]
  );
};
```

## 4.3 Adaptador PostgreSQL – productos más vendidos

El orden se toma de una lista blanca de cláusulas fijas, nunca de texto del usuario. El resto de los valores van como parámetros (\$1...\$4).

```
const { rows } = await this.db().query(
  `SELECT
    pr.id AS producto_id,
    pr.nombre AS nombre,
    SUM(d.cantidad)::int AS unidades,
    SUM(d.cantidad * d.precio_unitario) AS ingresos,
    COUNT(DISTINCT d.pedido_id)::int AS pedidos
  FROM detalle_pedido d
  INNER JOIN pedidos p ON p.id = d.pedido_id
  INNER JOIN productos pr ON pr.id = d.producto_id
  WHERE ${FILTRO_PEDIDOS}
  GROUP BY pr.id, pr.nombre
  ORDER BY ${orden}
  LIMIT $4`,
  [filtro.desde, filtro.hasta, filtro.estados, limite]
);
```

## 4.4 Caso de uso – ticket promedio y porcentajes por estado

```
const construir = r => ({
  ingresos: dinero(r.ingresos),
  pedidos: Number(r.pedidos || 0),
  clientesCompradores: Number(r.clientes || 0),
  unidadesVendidas: Number(r.unidades || 0),
  ticketPromedioPedido: dinero(dividir(r.ingresos, r.pedidos)),
  ticketPromedioCliente: dinero(dividir(r.ingresos, r.clientes))
});
```

```
const total = Object.values(acumulado).reduce((s, x) => s + x.cantidad, 0);

return {
  rango: this.describirRango(filtro),
  total,
  estados: CATEGORIAS.map(c => ({
    clave: c.clave,
    etiqueta: c.etiqueta,
    cantidad: acumulado[c.clave].cantidad,
    monto: dinero(acumulado[c.clave].monto),
    porcentaje: porcentaje(dividir(acumulado[c.clave].cantidad, total) * 100)
  })),
}
```

## 4.5 Rutas protegidas y composición

```
// Todas las rutas requieren token y rol admin.
router.use(verificarToken, autorizarRoles("admin"));

// Panel completo en una sola petición
router.get("/dashboard", analyticsController.dashboard.bind(analyticsController));

// Métricas individuales
router.get("/resumen", analyticsController.resumen.bind(analyticsController));
router.get("/ingresos", analyticsController.ingresos.bind(analyticsController));
router.get("/productos-top", analyticsController.productosTop.bind(analyticsController));
router.get("/estados-pedidos", analyticsController.estadosPedidos.bind(analyticsController));
```

```
const analyticsRepository =
  new AnalyticsRepositoryAdapter();

const analyticsService =
  new AnalyticsService(analyticsRepository);

const analyticsController =
  new AnalyticsController(analyticsService);

app.use(
  "/api/reportes",
  createAnalyticsRoutes(analyticsController)
);
```

## 4.6 Frontend – carga del panel y gráfico circular (Recharts)

Al cambiar un filtro (rango, granularidad, base, límite u orden) se hace una sola petición a /dashboard. Un contador descarta respuestas viejas si el usuario cambia filtros rápido.

```
const respuesta = await getDashboard({
  desde: rango.desde,
  hasta: rango.hasta,
  granularidad: granularidad === "auto" ? undefined : granularidad,
  base,
  limite,
  ordenarPor
});
```

```
<PieChart>
  <Pie
    data={conDatos}
    dataKey="cantidad"
    nameKey="etiqueta"
    innerRadius={58}
    outerRadius={90}
    paddingAngle={2}
    stroke="ffffff"
    strokeWidth={2}
    onMouseEnter={(_, i) => setActivo(conDatos[i]?.clave)}
    onMouseLeave={() => setActivo(null)}
  >
    {conDatos.map(e => (
      <Cell
        key={e.clave}
        fill={COLORES_ESTADO[e.clave]}
        opacity={activo && activo !== e.clave ? 0.35 : 1}
      />
    ))}
  </Pie>
  <Tooltip content={<TooltipEstado />} />
</PieChart>
```

## 5. Verificación

### 5.1 Pruebas unitarias (npm test)

El caso de uso se prueba con un repositorio falso en memoria (sin PostgreSQL). También corren las pruebas de la actividad anterior (notificaciones).

```
ok 1 - sin fechas usa los últimos 30 días y granularidad diaria
ok 2 - sugiere semana o mes según la longitud del rango
ok 3 - base 'todos' incluye pendientes y excluye cancelados
ok 4 - el período anterior tiene la misma duración y termina un día antes
ok 5 - rechaza filtros inválidos con ErrorFiltro
ok 6 - calcula tickets promedio y variación contra el período anterior
ok 7 - sin pedidos los tickets son 0 y la variación no divide entre cero
ok 8 - crecer desde cero devuelve variación null (sin base de comparación)
ok 9 - agrupa estados de BD en Pendiente/Pagado/Enviado/Cancelado con porcentajes
ok 10 - el ranking respeta el límite y calcula participación y precio promedio
ok 11 - valida límite y orden del ranking
ok 12 - la serie de ingresos reporta total, promedio y mejor período
ok 13 - el dashboard reúne todas las métricas en una respuesta
ok 14 - al crear un pedido se envía comprobante al cliente y aviso al admin
ok 15 - si el correo falla, el pedido se registra de todos modos
ok 16 - sin adaptador de correo el pedido se crea sin notificar
ok 17 - un adaptador que no implementa el puerto lanza error
ok 18 - la plantilla del cliente incluye folio, desglose, total y datos bancarios
ok 19 - la plantilla del administrador incluye cliente y total
# tests 19
# pass 19
# fail 0
```

### 5.2 Validación de las consultas SQL

El adaptador se ejecutó contra PostgreSQL 16 con el mismo esquema del proyecto y 220 pedidos de demostración (npm run seed:analitica). Los resultados se cotejaron con consultas SQL independientes. Por ejemplo, para septiembre de 2026 (solo pagados):

**\$956,015.00**

Ingresos (servicio = SQL  
directo)

**53**

Pedidos pagados

**\$18,038.02**

Ticket promedio por  
pedido

**89 u.**

Producto #1 en unidades

Con el script de índices (docs/sql/indices-analitica.sql), el plan de ejecución usa Bitmap Index Scan on idx\_pedidos\_fecha\_estado para el filtro de fecha y estado.

## 6. Entregable 3 – Demostración visual del dashboard

---

Panel ejecutado en <http://localhost:5173> con sesión de administrador → menú **Reportes**.

### Captura 1

#### Panel completo con datos

KPIs con variación, tendencia de ingresos, dona de estados y ranking (últimos 30 días).

### Captura 2

#### Gráfico interactivo de ingresos

Tooltip al pasar el cursor y cambio entre líneas y barras.

### Captura 3

#### Gráfico circular de estados

Distribución porcentual Pendiente / Pagado / Enviado / Cancelado.

**Captura 4**

**Ranking de productos**

Top 10 ordenado por monto recaudado.

**Captura 5**

**Filtro "Este año"**

Serie agrupada por mes al cambiar el rango.

**Captura 6**

**Rango personalizado**

Selección de fechas Desde / Hasta y botón Aplicar.

## 7. Ejecución local

---

```
cd ~/webapp
git pull
npm install
npm test
npm run seed:analitica
npm run dev

# rama main
# backend
# 19 pruebas
# datos demo (opcional)
# backend -> http://localhost:3000

cd frontend
npm install
npm run dev

# instala recharts
# frontend -> http://localhost:5173

# Opcional: índices para acelerar los reportes
sudo -u postgres psql -d ecommerce_db -f docs/sql/indices-analitica.sql

# Quitar los datos demo
npm run seed:analitica -- --limpiar
```

## 8. Conclusión

---

El subsistema analítico se integró como un módulo hexagonal independiente. AnalyticsService concentra las reglas de las métricas (tickets, porcentajes, comparación de períodos) y solo depende del puerto AnalyticsRepository. El adaptador PostgreSQL resuelve las agregaciones donde son eficientes, en el motor de base de datos. Las entidades operativas y sus repositorios no cambiaron. El panel en React + Recharts ofrece filtros temporales y gráficas interactivas para apoyar la toma de decisiones del administrador.

CÓDIGO FUENTE

<https://github.com/LuisAGA-1/ecommerce-5m>